
```

#ifndef LIST_H
#define LIST_H
/* List.h
 *
 * doubly-linked, double-ended list with Iterator interface
 * Project UID c1f28c309e55405daf00c565d57ff9ad
 * EECS 280 Project 4
 */

#include <iostream>
#include <cassert> //assert
#include <cstddef> //NULL

template <typename T>
class List {
    //OVERVIEW: a doubly-linked, double-ended list with Iterator interface
public:
    List() {
        list_size = 0;
        first = nullptr;
        last = nullptr;
    }
    List(const List &a) {
        //if (*this != a) {
            list_size = 0;
            first = nullptr;
            last = nullptr;
            this->copy_all(a);
        /*
            first = a.first;
            last = a.last;
            list_size = a.list_size;
        */
    }
    ~List() {
        this->clear();
    }

    // construct an Iterator at a specific position
    /*
    //EFFECTS: returns true if the list is empty
    bool empty() const;

    //EFFECTS: returns the number of elements in this List
    //HINT:    Traversing a list is really slow.  Instead, keep track of the size
    //         with a private member variable.  That's how std::list does it.
    int size() const;

    //REQUIRES: list is not empty
    //EFFECTS: Returns the first element in the list by reference
    T & front();

    //REQUIRES: list is not empty
    //EFFECTS: Returns the last element in the list by reference
    T & back();

```

```

//EFFECTS: inserts datum into the front of the list
void push_front(const T &datum);

//EFFECTS: inserts datum into the back of the list
void push_back(const T &datum);

//REQUIRES: list is not empty
//MODIFIES: may invalidate list iterators
//EFFECTS: removes the item at the front of the list
void pop_front();

//REQUIRES: list is not empty
//MODIFIES: may invalidate list iterators
//EFFECTS: removes the item at the back of the list
void pop_back();

//MODIFIES: may invalidate list iterators
//EFFECTS: removes all items from the list
void clear();

// You should add in a default constructor, destructor, copy constructor,
// and overloaded assignment operator, if appropriate. If these operations
// will work correctly without defining these, you can omit them. A user
// of the class must be able to create, copy, assign, and destroy Lists
*/

bool empty() const {
    if (list_size == 0) {
        return true;
    }
    else {
        return false;
    }
}

//EFFECTS: returns the number of elements in this List
//HINT: Traversing a list is really slow. Instead, keep track of the size
// with a private member variable. That's how std::list does it.
int size() const {
    return list_size;
}

//REQUIRES: list is not empty
//EFFECTS: Returns the first element in the list by reference
T & front() {
    //Node *first_elm = first;
    return first->datum;
}

//REQUIRES: list is not empty
//EFFECTS: Returns the last element in the list by reference
T & back() {
    return last->datum;
}

//EFFECTS: inserts datum into the front of the list
void push_front(const T &datum){
    Node* new_node = new Node();
    new_node->datum = datum;

```

```

    new_node->next = first;
    new_node->prev = nullptr;
    if (first != nullptr) {
        first->prev = new_node;
    }
    else {
        last = new_node;
    }
    first = new_node;
    list_size++;
}

//EFFECTS: inserts datum into the back of the list
void push_back(const T &datum) {
    //std::cout << "Entering push back about to create a new node" << std::endl;
    Node* new_node = new Node();
    //std::cout << "Created a new node " << datum << std::endl;
    new_node->datum = datum;
    new_node->prev = last;
    new_node->next = nullptr;
    //std::cout << "Set some pointers" << std::endl;
    if (last != nullptr) {
        last->next = new_node;
    }
    else {
        first = new_node;
    }
    last = new_node;
    //std::cout << "About to increment list size" << std::endl;
    list_size++;
}

//REQUIRES: list is not empty
//MODIFIES: may invalidate list iterators
//EFFECTS: removes the item at the front of the list
void pop_front() {
    Node* to_delete_node = first;
    if (to_delete_node->next != nullptr) {
        Node* second_node = to_delete_node->next;
        second_node->prev = nullptr;
        first = second_node;
    }
    else {
        last = nullptr;
        first = nullptr;
    }
    free(to_delete_node);
    list_size--;
}

//REQUIRES: list is not empty
//MODIFIES: may invalidate list iterators
//EFFECTS: removes the item at the back of the list
void pop_back() {
    Node* to_delete_node = last;
    if (to_delete_node->prev != nullptr) {
        Node* second_node = to_delete_node->prev;
        second_node->next = nullptr;
        last = second_node;
    }
}

```

```

    }
    else {
        last = nullptr;
        first = nullptr;
    }
    free(to_delete_node);
    list_size--;
}

//MODIFIES: may invalidate list iterators
//EFFECTS: removes all items from the list
void clear() {
    while (!empty()) {
        pop_front();
    }
}

//REQUIRES: list is empty
//EFFECTS: copies all nodes from other to this
void copy_all(const List<T> &other) {
    List<int>::Iterator o_ptr = other.begin();
    //List<int> my_list;
    for (int i = 0; i < other.size(); i++) {
        //std::cout << "In copy all record " << i << " pushing back " << *o_ptr <<
std::endl;
        this->push_back(*o_ptr);
        //std::cout << "Finished pushing back about to increment o_ptr" << std::endl;
        ++o_ptr;
        //std::cout << "Finished incrementing o_ptr" << std::endl;
    }
}

private:
    //a private type

    int list_size = 0;

    struct Node {
        Node *next;
        Node *prev;
        T datum;
    };

    /*
List() {
    //std::cout << "Default Constructor" << std::endl;
}
List(const List &a) {

}
~List() {}
*/

```

```

Node *first;    // points to first Node in list, or nullptr if list is empty
Node *last;    // points to last Node in list, or nullptr if list is empty

```

```

//EFFECTS: returns true if the list is empty

```

```

public:
    //////////////////////////////////////////
    class Iterator {
        //OVERVIEW: Iterator interface to List

        // You should add in a default constructor, destructor, copy constructor,
        // and overloaded assignment operator, if appropriate. If these operations
        // will work correctly without defining these, you can omit them. A user
        // of the class must be able to create, copy, assign, and destroy Iterators.

        // Your iterator should implement the following public operators: *,
        // ++ (prefix), default constructor, == and !=.

public:
    friend struct Node;
    Iterator() {
        node_ptr = nullptr;
    }

    // construct an Iterator at a specific position
    Iterator(Node *p) {
        node_ptr = p; // return Iterator(p); // Not sure about this
    }

    // This operator will be used to test your code. Do not modify it.
    // Requires that the current element is dereferenceable.
    // Iterator(node_ptr = nullptr);
    // Iterator() node_ptr = nullptr;

    Iterator& operator--() {
        assert(node_ptr);
        node_ptr = node_ptr->prev;
        return *this;
    }

    Iterator& operator++() {
        assert(node_ptr);
        node_ptr = node_ptr->next;
        return *this;
    }
    inline bool operator==(const Iterator &a) const {
        if (&a != this) { // changed from -> to . instead of datum use node_ptr
            return false;
        }
        else {
            return true;
        }
    }
    inline bool operator!=(const Iterator &a) const {
        return !(&a == this);
    }
    T operator*() {
        return node_ptr->datum; // was *this->datum
    }

private:
    Node *node_ptr; //current Iterator position is a List node

```

```

// add any additional necessary member variables here

// add any friend declarations here
friend class List;
friend struct Node;
//Iterator();

}; //List::Iterator
////////////////////////////////////

// return an Iterator pointing to the first element
Iterator begin() const {
    //node_ptr = first;
    return Iterator(first);
}

// return an Iterator pointing to "past the end"
Iterator end() const {
    return Iterator(nullptr);
}

//REQUIRES: i is a valid, dereferenceable iterator associated with this list
//MODIFIES: may invalidate other list iterators
//EFFECTS: Removes a single element from the list container
void erase(Iterator i) {
    Node* prev_node = i.node_ptr->prev; // changed from -> to .
    Node* next_node = i.node_ptr->next; // changed from -> to .
    if (next_node == nullptr) {
        pop_back();
    }
    else if (prev_node == nullptr) {
        pop_front();
    }
    else {
        prev_node->next = next_node;
        next_node->prev = prev_node;
        // free(i); //commenting out this for now
        list_size--;
    }
}

//REQUIRES: i is a valid iterator associated with this list
//EFFECTS: inserts datum before the element at the specified position.
void insert(Iterator i, const T &datum) {
    Node* prev_node;
    if (i.node_ptr == nullptr) {
        push_front(datum);
    }
    else {
        prev_node = i.node_ptr->prev;
        if (prev_node == nullptr) {
            push_front(datum);
        }
        else {
            Node* new_node = new Node();
            new_node->datum = datum;
            new_node->next = prev_node->next; // changed from = i to = i.node_ptr->next
            new_node->prev = prev_node;
        }
    }
}

```

```

        prev_node->next = new_node;
        i.node_ptr->prev = new_node;
        list_size++;
        /*
        Node* new_node = new Node();
        new_node->datum = datum;
        new_node->next = i.node_ptr->next; // changed from = i to = i.node_ptr-
>next
        new_node->prev = prev_node;
        prev_node->next = new_node;
        i.node_ptr->prev = new_node;
        list_size++;
        */
    }
}
}
/*
inline bool operator==(Iterator& a, Iterator& b) {
    if (a->datum != b->datum) {
        return false;
    }
    else {
        return true;
    }
}
inline bool operator!=(Iterator& a, Iterator& b) {
    return !(a == b);
}
*/

}; //List

```

```

////////////////////////////////////
// Add your member function implementations below or in the class above
// (your choice). Do not change the public interface of List, although you
// may add the Big Three if needed. Do add the public member functions for
// Iterator.

```

```

#endif // Do not remove this. Write all your code above this line.

```